MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 09

REPORT ON: TASK REPORT

TASK NO: 01-03

TASK 1 – ADVANCED CONDITIONAL LOGIC & FLOW DESIGN

Introduction

This task focuses on understanding advanced conditional logic in Kotlin and designing structured decision flows for application features. Control flow structures such as **if-else statements, when expressions, and conditional branching** help developers make decisions in an application. Proper planning of logic improves readability, maintainability, and debugging of applications.

Key Concepts

If–Else Statement

An if–else statement executes different blocks of code depending on a condition.

```
val amount = 120

if (amount > 100) {
   println("High expense")
} else {
   println("Normal expense")
}
```

If–Else Expression

In Kotlin, if can also return a value.

```
val amount = 50
val category = if (amount > 100) "High" else "Low"
println(category)
```

When Statement

The when statement is used as an alternative to multiple if–else conditions.

```
val day = 3

when (day) {
    1 -> println("Monday")
    2 -> println("Tuesday")
    3 -> println("Wednesday")
    else -> println("Other day")
}
```

When Expression with Range and Type Check

```
val value = 15

when (value) {
    in 1..10 -> println("Small")
    in 11..20 -> println("Medium")
    else -> println("Large")
}
```

Application Feature Description

Feature: **Expense Tracker Categorization**

This feature categorizes transactions based on the amount entered by the user. It validates the amount and determines whether a discount or surcharge should be applied.

Decision Flow Table

Condition	Result
amount < 0	Reject transaction
amount in 0..100	Category A
amount > 100 and discountEligible	Apply discount
otherwise	Apply surcharge

Decision Flow Diagram

```
                User enters amount
                        |
                        v
              Is amount < 0 ?
                 /          \
              Yes            No
            Reject      Is amount <=100 ?
               /                  \
            Yes                    No
      Category A         Discount Eligible?
                            /        \
                         Yes         No
           Apply Discount     Apply Surcharge
```

Compose Integration Plan

The decision logic can be integrated into Jetpack Compose using state variables. The amount entered by the user can be stored using remember { mutableStateOf() }. Based on the value entered, conditional logic determines the category or action. The UI updates dynamically by displaying error messages, category labels, or calculated values.

Reflection

Planning logic before coding helps developers understand decision paths clearly. Complex nested conditions can make the code difficult to maintain. Using Kotlin expressions such as when and structured conditionals improves readability and simplifies debugging.

TASK 2 – NULLABILITY, SAFE CALLS & ROBUST DATA HANDLING

Introduction

This task focuses on Kotlin's null safety system, which helps prevent runtime crashes caused by null values. Developers learn how to handle nullable and non-nullable types safely while building reliable Android applications.

Key Concepts

Nullable Type

A nullable type can store a null value.

Example:

```
val name: String? = null
```

Non-Nullable Type

A non-nullable type cannot contain null.

```
val name: String = "Sahil"
```

Safe Call Operator (?.)

val length = name?.length

Elvis Operator (?:)

val length = name?.length ?: 0

Non-null Assertion (!!)

val length = name!!.length

Data Handling Scenario

Scenario: **User Profile Fetch from Server**

In an application, user profile data is fetched from a server. Some fields such as **profile image, nickname, or email** may be missing or null due to incomplete user data.

Null Handling Logic Table

Field	Type	Nullable	Handling Strategy	UI Fallback
nickname	String?	Yes	Use Elvis operator	"Guest User"
photoUrl	String?	Yes	Check before loading image	Default avatar
email	String?	Yes	Display if available	"Not provided"

Compose UI Handling Example

Text(text = user.nickname ?: "Guest User")

If the nickname is null, the UI automatically displays **Guest User**.

Reflection

Null safety is important in Android development because many data sources such as APIs and databases may return null values. Without proper null handling, applications may crash due to NullPointerExceptions. Kotlin's null safety features improve reliability and help developers write safer code.

TASK 3 – FEATURE IMPLEMENTATION STRATEGY (CONDITIONALS + NULLABILITY)

Introduction

This task integrates conditional logic and null safety to design a medium-complex application feature. The feature includes user inputs, decision making, and dynamic UI updates using Compose.

Feature Description

Feature: **Event RSVP Screen**

Users enter the number of guests, select a meal preference, and choose whether to accept or decline the invitation. Based on the inputs, the application determines whether the submit button should be enabled or disabled.

Data and State Model

Field	Type
guests	Int?
mealPreference	String?
rsvpStatus	Boolean

Compose State Example:

```
var guestsInput by remember { mutableStateOf("") }
var mealPreference by remember { mutableStateOf<String?>(null) }
var rsvpStatus by remember { mutableStateOf(false) }
```

Skeleton Logic

```
val guestsCount = guestsInput.toIntOrNull()

if (rsvpStatus) {

    if (guestsCount != null && guestsCount > 0) {
        // Enable submit
    } else {
        // Show error
    }

} else {
    // Decline message
}
```

Default value example:

```
val meal = mealPreference ?: "Standard"
```

Edge Cases and Test Planning

Case	**Expected Result**
Guests field empty	Show error message
Guests = 0	Disable submit
Meal preference null	Use default value
Rapid toggle changes	UI updates correctly

Reflection

Combining conditional logic and null safety increases the complexity of application design. Developers must carefully manage state, inputs, and edge cases to avoid errors. Using clear logic structures, default values, and proper validation helps reduce complexity and makes applications easier to maintain.